

User Manual

EduShield: Cybersecurity Monitoring System

Year: 2026

Semester: Summer 2026

Project leader: 65011 - Nimra Naeem

Member 2: 63744 - Maimuna Rahman Muna

Member 3: 62810 - Md Ashraf Hossain

Member 4: 62863 - Md Hasibul Hasan Khan Shanto

Table of Contents

1. SCOPE	3
1.1 IDENTIFICATION	3
1.2 SYSTEM OVERVIEW	3
1.3 DOCUMENT OVERVIEW	3
2. DESIGN DECISIONS	3
2.1 TECHNOLOGY STACK RATIONALE	3
2.2 ARCHITECTURE RATIONALE	3
3. SYSTEM ARCHITECTURE	4
4. DETAILED DESIGN	5
4.1 FRONTEND DASHBOARD	5
4.1.1 Page Structure	5
4.2 FASTAPI BACKEND	6
4.3 SCANNING ENGINE	6
4.4 PDF REPORT ENGINE	9
5. NOTES	9
5.1 GLOSSARY	9
5.2 REFERENCES	10

Table of Figures

Figure 3.1: EduShield System Architecture	4
Figure 4.2: Flow Diagram for reconnaissance module	6
Figure 4.3: Flow Diagram for SSL/TLS Analyzer Module	7
Figure 4.4: Flow Diagram for Header Analyzer Module	7
Figure 4.5: Flow Diagram for Login Scanner	8
Figure 4.6: Flow Diagram for Phishing Detector	8
Figure 4.7: Flow Diagram for OWASP ZAP Scanner	9

1. Scope

1.1 Identification

This is a user-manual of EduSheild platform which is a Cyber-security monitoring system build for Australian educational institutes. This has been a 12-week internship project of VIT MITS students' team with Tech Adaptive Pandit Pty Ltd. Adelaide, SA.

1.2 System Overview

We developed a web application based cyber-security monitoring platform called EduSheild. It contains six scanner modules to analyse the cyber threats, vulnerabilities & check whether it follows Australian Compliance and then display to the dashboard. Following that, the system generates a pdf report containing all the findings and recommendations.

1.3 Document Overview

The user manual contains the following sections:

Section	Content
Scope	Description of the system identification, system
Design Decisions	Highlights the technical and logical reasons for choosing the technologies
System Architecture	Illustrates the system architecture
Detailed Design	Detailed explanation of frontend, javascript functions, API endpoints, scanning modules and PDF report generation
Notes	Lists of Glossary and references

2. Design Decisions

2.1 Technology Stack Rationale

Plain HTML, CSS & JS has been chosen for frontend as we have limited timeframe and to have smooth integration which doesn't require any complex setup.

The backend was selected to be FastAPI as it is lightweight and fast and can be easily integrated with the frontend and scanning modules. It can also work with several scanner modules in parallel at the same time, which is important for maintaining a system's proper asynchronisation; and it's efficient for creating APIs. Another reason was Python was the language of choice for most Cybersecurity scanning tools and libraries, making implementation easier.

2.2 Architecture Rationale

The development process has been broken down into different components – EduShield system. User interaction is handled by the frontend, API communication and scan lifecycle by

the backend, scanner modules are used to do the actual security analysis, and the PDF engine generates reports.

3. System Architecture

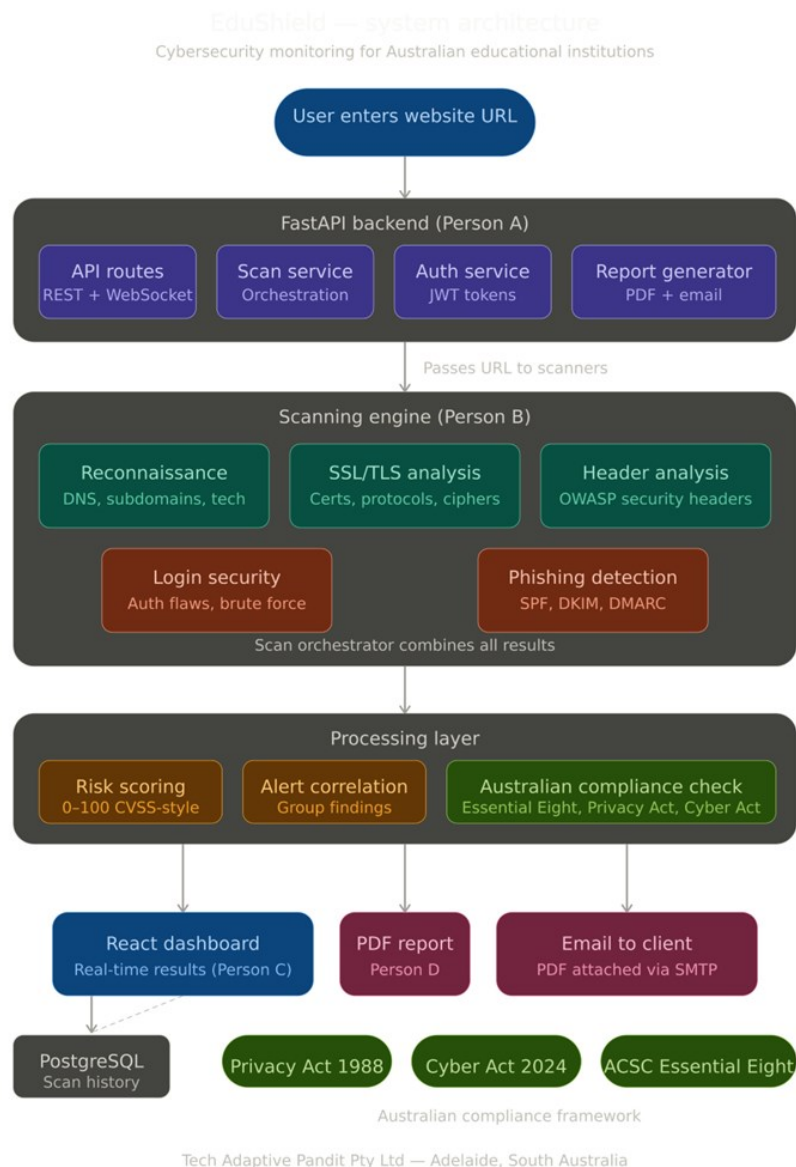


Figure 3.1: EduShield System Architecture

Requests from the Frontend dashboard are passed to the Backend (FastAPI), and this then communicates with the scanning engine to perform scans. The processing layer takes the risk score and compliance results before they are presented on the dashboard and PDF report.

4. Detailed Design

4.1 Frontend Dashboard

The aim of developing the dashboard was to be simple, easy to understand and smooth integration with backend. So, we agreed to create the dashboard using plain HTML, CSS and JavaScript. The file structured in three separate files that located in the same folder “EduSheild”. The main structure of the page is in index.html, the style contains in the file style.css and script.js contains all the JavaScript logic for navigation, API integration, chart generating and fallback simulation.

4.1.1 Page Structure

The dashboard has six pages: New Scan, Scan Results, Threat Analysis, Recommendation, AU Compliance & Scan History.

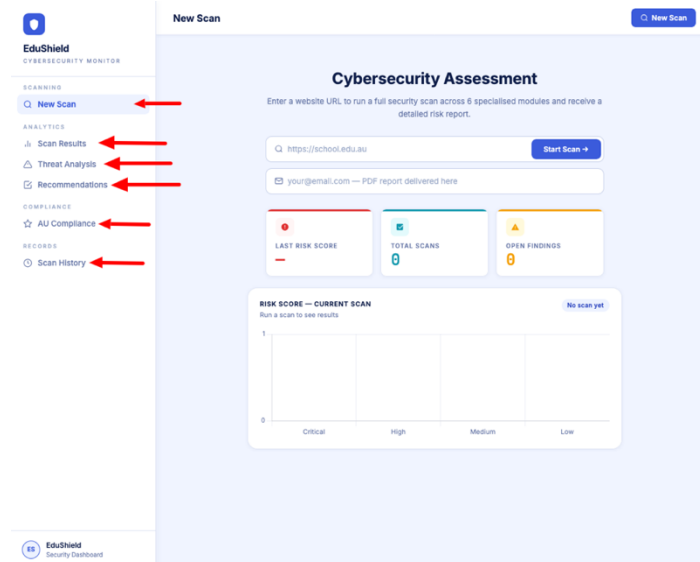


Figure 4.1: Six pages of the dashboard

a. New Scan Page

New scan page has the input fields for URL & email. The scanners will fetch the URL from this field, scan the web threats and send the report to the specific email address.

b. Scan Result

This page displays the risks score, severity of the risks and the findings from the scanner modules. Pie chart displays the severity of the risk and findings has used bar chart to display to the dashboard.

c. Threat Analysis

Threat analysis page displays the type of threats and findings in two different pie charts. However, the detailed finding has been stored in a table down to the page for proper understanding.

d. Recommendations

One of the most important pages is Recommendation. This page highlights the priorities of each threat, when it should fix and how hard the fixes with a detailed table of recommendations.

e. AU Compliance

This page displays that the checked page/website/webpage follows the Australian Compliance and the percentages of it.

f. Scan History

Scan history page has the table of scans has been run on the current session, with the URL, date and status.

4.2 FastAPI Backend

The FastAPI backend is the central point for managing operations in the EduShield platform. It's the one that asks Frontend dashboard for scan requests, runs the necessary scanner modules, and gathers the results. Risk scores, severity, and compliance information are also calculated on the back end and fed back to the front end. Moreover, it has PDF report generation and e-mail delivery features.

4.3 Scanning Engine

This scanning engine receives the targeted URL from the backend and then start processing each scanning module individually. Each module checks different security domain such as Reconnaissance, SSL/TLS Analyzer, Header Analyser, login Scanner, Phishing Detector and vulnerabilities in OWASP ZAP. After the scanning, they return the result in JSON format and sent it back to backend where the final security report is shown to the user.



Figure 4.2: Flow Diagram for reconnaissance module

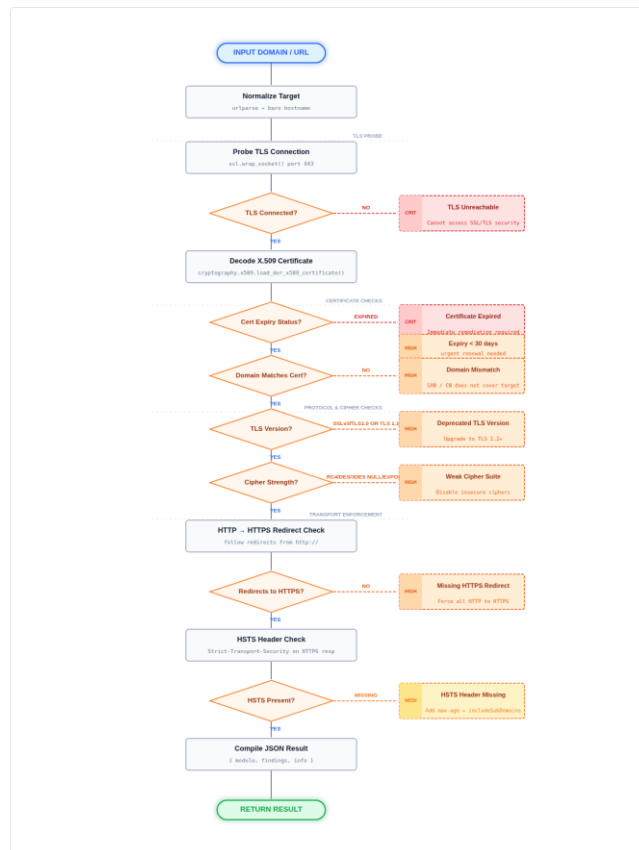


Figure 4.3: Flow Diagram for SSL/TLS Analyzer Module

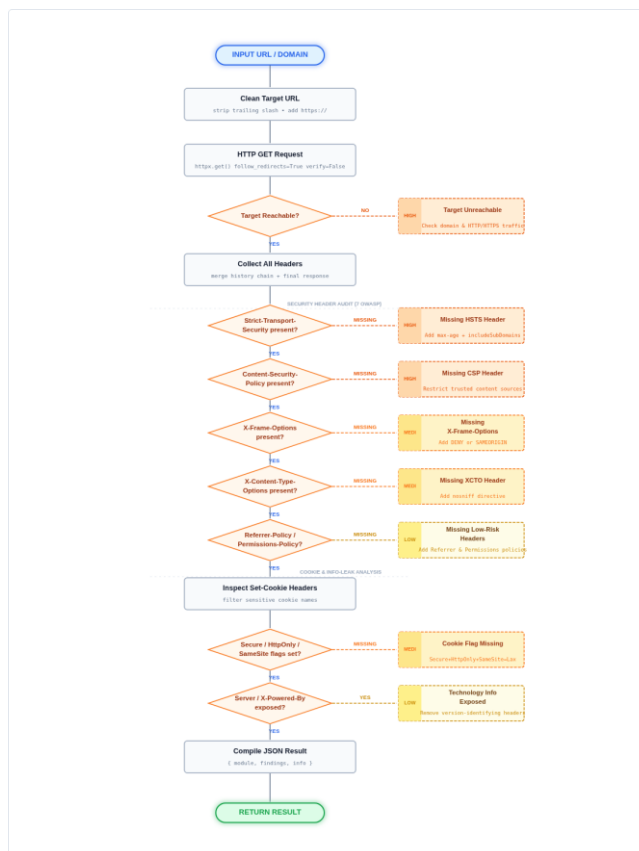


Figure 4.4: Flow Diagram for Header Analyzer Module



Figure 4.5: Flow Diagram for Login Scanner

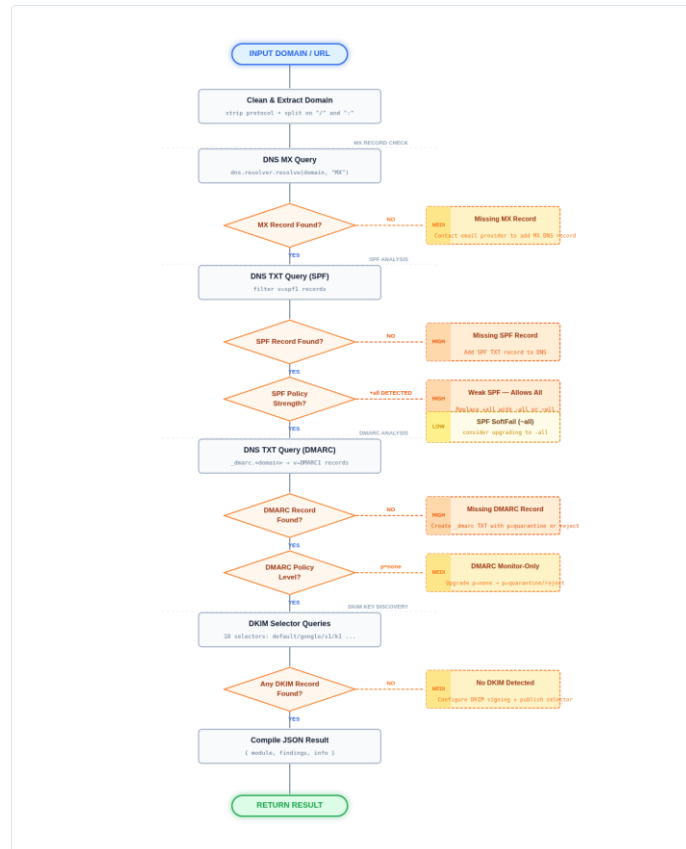


Figure 4.6: Flow Diagram for Phishing Detector

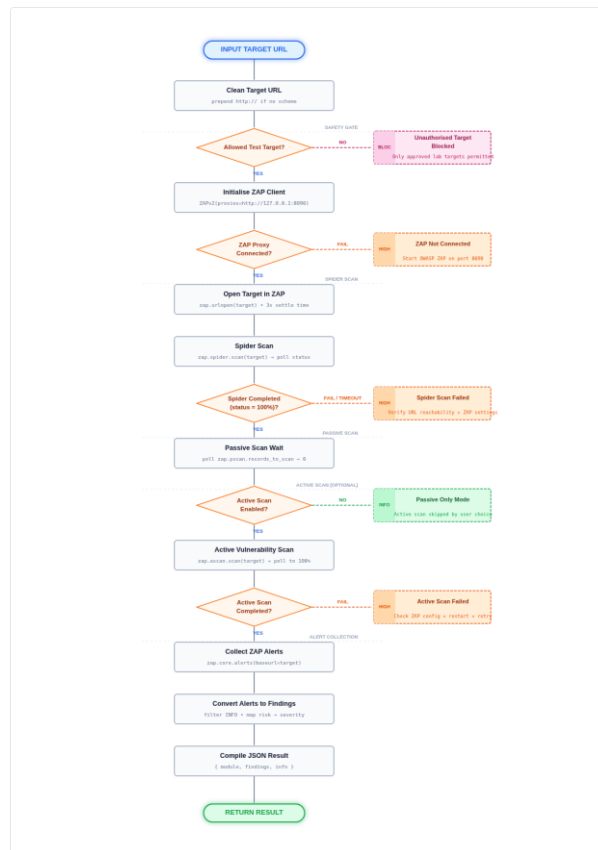


Figure 4.7: Flow Diagram for OWASP ZAP Scanner

4.4 PDF Report Engine

A professional cybersecurity report is generated after each scan with the pdf report engine. It categorises the results of the scan, severity summary, recommendations, and compliance results in a systematic PDF format. The back end processes the created report and outputs a file that can be downloaded and will also e-mail it to the user if they have entered an e-mail address.

5. Notes

5.1 Glossary

Term	Meaning
API	Stands for Application Programming Interface, and it is used for the communication between the Frontend and Backend
FastAPI	It is a backend framework of Python, which is used to manage the requests of AI and scan the operations
Backend	It is a component of the server-side, which is used to process the data and manage the system's functions
PDF Report	This contains the documents of scan findings and recommendations

SMTP	Simple Mail Transfer Protocol is used to send email
Database	To save scan history and results used storage system
Scanner Engine	It is a main security component of this project. It receives the website URL, run the scanner modules and return the final scan result to the backend.
Scanner Module	An individual security checker that tests one specific area of the website, such as SSL/TLS, headers, login pages, or phishing protection.
Finding	This is the observation by individual scanner modules during the scan.
Severity Level	It is rating defined for risks for each finding, such as Critical, High, Medium, Low, or Info

5.2 References

- [1] S. Ramírez, “FastAPI Documentation,” FastAPI, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [2] Python Software Foundation, “Python Documentation,” Python.org, 2026. [Online]. Available: <https://docs.python.org/3/>
- [3] Australian Cyber Security Centre, “Essential Eight Maturity Model,” ACSC, 2026. [Online]. Available: <https://www.cyber.gov.au/>
- [4] Jinja Project, “Jinja Documentation,” Pallets Projects, 2026. [Online]. Available: <https://jinja.palletsprojects.com/>
- [5] CourtBouillon, “WeasyPrint Documentation,” WeasyPrint, 2026. [Online]. Available: <https://weasyprint.org/>